

Machine Learning Classification:
Differentiating Bacterial Exotoxins based on
Exotoxin Types and Targets

Didem Dost

February 13, 2025

Acknowledgements

Abstract

Contents

1	Introduction	5
1.1	Bacterial Exotoxins	5
1.2	Motivation	6
1.3	Objectives	7
1.4	Structure of Thesis	7
2	Background	9
3	Methods	10
3.1	Data Collection and Curation	11
3.1.1	Dataset Structure and Preprocessing	12
3.1.2	Dataset Composition	12
3.2	Descriptive Statistical Analysis of Sequences	13
3.2.1	Mean Sequence Length	13
3.2.2	Hydrophobicity Analysis	13
3.3	Redundancy Reduction	14
3.4	Prediction of 3D Protein Folds	15
3.4.1	Extraction of Confidence Scores from AlphaFold2 Predictions	15
3.5	Preparation of Training and Test Datasets	16
3.5.1	Method 1: Similarity-Based Data Splitting	16
3.5.2	Method 2: Random Splitting After Redundancy Reduction	16
3.6	Embedding Generation	16
3.7	Dimensionality Reduction	17
3.8	Machine Learning Model Training and Evaluation	17
3.8.1	Model Selection	17
3.8.2	Additional Approach: Hierarchical Predictor	18

3.8.3	Additional Approach: BLAST-Based Predictor	19
3.8.4	Negative Control: Scrambled Protein Sequence Em- beddings	19
4	Results	20
4.0.1	Data Exploration and Preprocessing	20
4.0.2	Redundancy Reduction	22
4.0.3	Feature Analysis of Exotoxins	22
4.0.4	Quality of Protein Folds Based On pLDDT Scores . . .	27
4.0.5	Dimensionality Reduction Techniques	29
4.0.6	ExoType Predictor	33
4.0.7	Model Training	33
4.0.8	Performance Evaluation	35
4.0.9	ExoTarget Predictor	45
4.0.10	Model Training	45
4.0.11	Performance Evaluation	47
5	Conclusion	51
6	Discussion	52
6.1	Additional Performance Metrics	53

Chapter 1

Introduction

1.1 Bacterial Exotoxins

Bacterial toxins are molecules produced by bacteria that can cause harm by interacting with host tissues or disrupting cellular processes. Some bacteria secrete extracellular enzymes that degrade host tissue or alter immune responses[1]. For example, diphtheria toxin prevents protein synthesis. Similarly, *S. aureus* can overstimulate immune responses, while cholera toxin modifies second messenger pathways to aid bacterial survival and transmission without directly invading host tissues. [2].

Bacterial toxin forms are classified into bacterial endotoxins and exotoxins. Endotoxins are lipopolysaccharides (LPS) found in the outer membrane of Gram-negative bacteria and are released upon bacterial cell lysis. Exotoxins, on the other hand, are proteins actively secreted into the extracellular environment by both Gram-positive and Gram-negative bacteria [3]. Therefore, exotoxins have several modes of action that allow the bacteria to manipulate the host.

Bacterial exotoxins are primarily classified as Type I, Type II, and Type III; although a fourth type has been proposed, it has not yet been fully recognized in the scientific literature. Type I exotoxins, mainly superantigens, do not enter host cells. Instead, they overstimulate the immune system by triggering excessive cytokine release, ~~which can~~ potentially ~~lead~~ leading to toxic shock syndrome. Type II exotoxins, known as membrane-disrupting toxins, function in two main ways. They either form pores in host cell membranes, causing osmotic imbalance and lysis, or they enzymatically degrade mem-

brane phospholipids, particularly in immune cells such as phagocytes. Type III toxins, commonly referred to as A-B toxins, consist of an enzymatic A subunit that disrupts intracellular processes and a B subunit that facilitates toxin entry by binding to specific host receptors. ~~On a final note,~~ The proposed Type IV exotoxins operate by degrading extracellular matrix. ~~These~~ In general, exotoxins contribute significantly to bacterial virulence by impairing host immune responses, damaging tissues, and facilitating bacterial survival and proliferation [4] [5]. ~~Overall toxin~~ , and their biosynthesis can be influenced by a variety of environmental and genetic factors[6].

1.2 Motivation

The advancement of high-throughput technologies has revolutionized bacterial toxin research by significantly reducing the cost of sequencing bacterial genomes. As bacterial genomes are being sequenced at an accelerating rate, the need for efficient annotation tools has emerged.[7][8] While trying to annotate a vast amount of sequences, wet-lab methods have been inadequate due to being time-consuming, costly and error-prone. Therefore, machine learning offers a promising computational alternative to overcome these limitations. Machine learning uses a variety of algorithms to learn from data by recognizing distinct patterns in it [9]. Data can be either unlabeled, which is used for unsupervised learning, or labeled, which makes it suitable for supervised learning.[10]. A method used for supervised learning is classification which divides the data in two classes. Traditional classification methods, a type of supervised learning, can be designed for binary or multiclass problems. The classification of bacterial exotoxins has garnered significant attention in recent years, particularly with the advent of machine learning (ML) techniques that promise enhanced accuracy and efficiency [11].

Several studies have leveraged machine learning for toxin classification, utilizing diverse models and methodologies [12][13]. However, most of ~~these~~ studies ~~them~~ have focused on general bacterial classification. They have also highlighted challenges for exotoxin classification, including small dataset sizes, issues with image quality, and the need for advanced feature extraction techniques [14].

Over the years, there still remains a significant gap in understanding if accurately distinguishing bacterial exotoxin among all types is possible. Machine learning classification methods have primarily relied on binary classifi-

cations such as toxin versus non-toxin or exotoxin versus endotoxin. [12][13][14]. Although current exotoxin research has been essential for understanding bacterial virulence factors, these methods often fail to provide insights into the distinct types of exotoxins based on their four mechanisms of action.

Furthermore, the research community has not yet developed any predictor that distinguishes bacterial exotoxins based on the organism it targets and invades. Motivated by these gaps in the existing literature, this thesis aims to overcome this gap by building novel predictors: ExoType and ExoTarget. ExoType predictors aim to distinguish among five types of exotoxins based on two types of inputs: protein sequences and protein folds. On the other hand, ExoTarget predictors are designed to classify the different targets of exotoxins based on two types of inputs: protein sequences and protein folds.”

1.3 Objectives

This study encompasses three objectives. The first objective is to develop ExoType, a predictor for classifying bacterial exotoxin types based on their mechanisms of action. It reviews five types of bacterial exotoxins that were selected based on their types: membrane-transducing (Type I), pore-forming (Type II), intracellular toxins (Type III), ECM-degrading (Type IV), and other types. The second is to predict the classification of bacterial exotoxin targets by building ExoTarget, if possible. This encompasses differentiating between targets of bacterial exotoxins, categorizing them into vertebrate and non-vertebrate targets. The final one is to inspect the best performing machine learning model for the classification of bacterial toxins based on types and targets. This objective investigates whether training models on bacterial exotoxin types and targets can achieve a competitive predictive performance.

This research uses protein sequences ~~and protein folds as raw input data~~ and transforms them to embeddings from pre-train models on primary sequences or folds (3D prediction structure), ~~providing a foundation for developing the most suitable predictor.~~ Although this is true, this is not an objective.

1.4 Structure of Thesis

~~I have modify this. When listing anything, you need to make it as a list or itemize. It makes it easy to be read and internalize that it is a listing.~~

- Chapter 1 introduces the key terms, outlines the motivation behind the thesis project, and provides an overview of the thesis structure.
- Chapter 2 provides background information, which includes a comprehensive review of bacterial exotoxins, machine learning classification systems and foundational machine learning techniques relevant to this study. It also reviews contributions to computational applications of bacterial exotoxins and explains how this study addresses a current gap in the field.
- Chapter 3 details the methodologies employed in this study. It covers data collection, preprocessing, feature extraction, model development, hyperparameter tuning, and model evaluation.
- Chapter 4 presents the results gathered from the feature analysis of protein sequences, protein embeddings, machine learning models including comparative performance metrics and visualizations.
- Chapter 5 analyzes the findings, compares them with existing literature, and explores their implications for machine learning applications in bacterial exotoxin research.
- Chapter 6 summarizes the study's contributions, addresses limitations, and proposes directions for future research.

Chapter 2

Background

Chapter 3

Methods

This study applies machine learning techniques to classify bacterial exotoxins based on their functional types and biological targets. The methodology follows a structured pipeline (scheme in 3.1) that involves data collection, preprocessing, feature extraction, and model training.

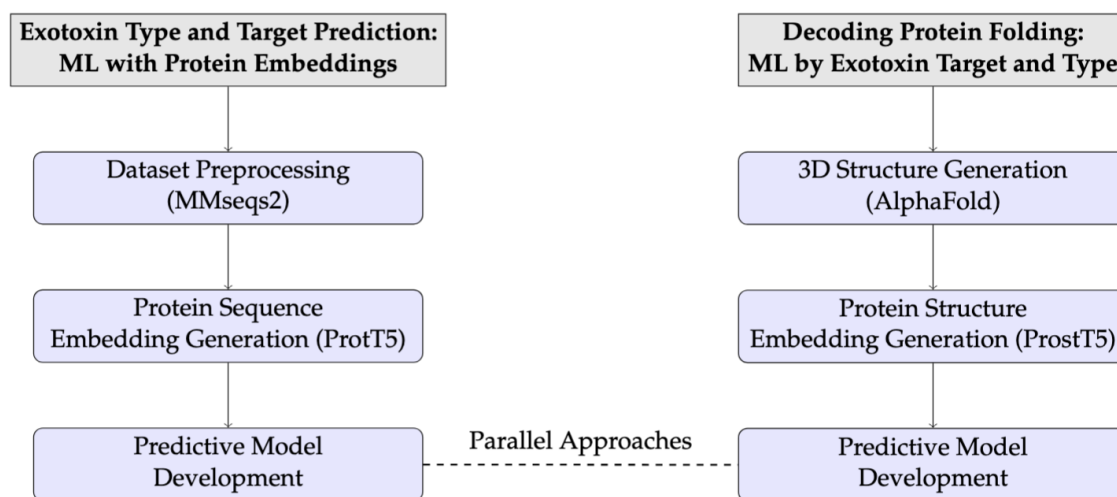


Figure 3.1: Overview of the key steps in the methodology: Exotoxin type and target predictions based on protein (left) and protein folding-based classification (right).

First, raw protein sequence

What is raw protein sequence? Usually a raw is the most basic, which in case of proteins will be the amino acid chemical structure. Please describe it as primary sequence or the traditional file for sequences like fasta files

data **is**

Usually methods are described in past tense. It is difficult for me to say here since I am not sure if you are just describing traditional pipelines or what you did in your thesis. Think about it when revising

collected and analyzed using descriptive statistics to understand key sequence characteristics. Then, the dataset is preprocessed by redundancy reduction techniques to derive it to a non-redundant dataset. Next, protein language models are used to generate numerical embeddings that capture meaningful sequence representations.

Following this, dimensionality reduction techniques are used to refine the embeddings. The processed embeddings are then used to train machine learning models, including conventional classifiers and hierarchical predictors. The machine learning models are evaluated using performance metrics for binary and multiclass classification tasks.

Figure 3.1 visualizes the key steps in building a machine learning classification pipeline for differentiating exotoxin types and targets. **I think this is not necessary if you keep my reference to the graphic I introduced in the first part of the section.**

3.1 Data Collection and Curation

Dear Didem, I thought we discussed this already. Yes, you are telling how the data was collected and curated, however the way you are declaring it here is as if you did it all by yourself. Please tell that the labeling was done by me, using the species associated with the toxin and the knowledge of the type of pathogenic association known in the literature. In science we all work and advance because other people have done previous work we use :-) Bacterial **toxins** protein sequences were collected from public sequence databases, such as UniProt and NCBI, **as decribed previously by Krueger et al. [15]** The dataset was curated **and labeled** by selecting proteins from bacterial pathogens based on their host specificity **by PD. Dr. Luisa F. Jimenez-Soto**. Proteins from bacterial pathogens affecting insects, plants, and other bacteria

were categorized as non-vertebrate targets, whereas proteins from bacterial pathogens affecting vertebrates—including fish, amphibians, birds, reptiles, and mammals—were classified as vertebrate targets.

Similarly, bacterial exotoxins were categorized based on their mode of action on how they exert their effect on the host. Bacterial exotoxins were classified into different functional types, including membrane-transducing toxins, pore-forming toxins, intracellular toxins, and toxins that degrade the extracellular matrix.

3.1.1 Dataset Structure and Preprocessing

The curated dataset was structured in a CSV file, which contains bacterial species, their exotoxin types, and their specific targets. This labeled data was used to annotate the input files to build a predictive model for classifying bacterial exotoxins.

The ~~raw~~ FASTA file stores each exotoxin ID alongside its corresponding protein sequence. To integrate these datasets, the CSV and FASTA files were merged using their IDs. The resulting fully labeled dataset includes bacterial exotoxin sequences along with their respective types and targets. This data preprocessing step is implemented in the `data-preprocessing.py` script using Python 3.13.1.

Then, a Python script was employed to eliminate sequences annotated as 'partial' or 'fragment' in their FASTA headers, thereby reducing incomplete sequences. This Python script is available as `data-cleaning.py`.

I love you put the files doing each task in here! you are making my life much easier and any other reader can go into the file if they want! Very OpenScieny :)

3.1.2 Dataset Composition

The final dataset contains 2,473 ~~data-points~~ sequences. Type III toxins have 2,148 entries, including both Type III and Type III Control. Type II toxins have 169 entries, including Type II and Type II PFT bacteria. Type I toxins have 42 entries, while Type IV toxins have 28 entries. An additional 104 entries are categorized as Unknown, as they do not belong to any of the four existing classes.

The target category consist of vertebrate-targeting exotoxins and non-vertebrate-targeting bacterial exotoxins, with 1,863 and 629 data points, re-

spectively.

3.2 Descriptive Statistical Analysis of Sequences

Understanding the statistical properties of protein sequences is essential for analyzing their biological characteristics and identifying potential biases in the dataset. Descriptive statistical analysis provides insights into fundamental sequence properties, such as length distributions and hydrophobicity.

This analysis was conducted using custom Python scripts, incorporating key libraries such as Biopython 1.85 for sequence handling, NumPy 2.2.2 for numerical computations, and Matplotlib 3.10.0 for visualization. This Python script can be accessed through `hydrophobicity-sequence-length.py` script.

3.2.1 Mean Sequence Length

Understanding length variability is crucial, as it may influence the training of machine learning models. The mean sequence length μ was calculated by using the following formula:

$$\mu = \frac{1}{N} \sum_{i=1}^N L_i \quad (3.1)$$

where:

- μ is the mean sequence length,
- N is the total number of sequences in the dataset,
- L_i represents the length of the i^{th} sequence.

By analyzing sequence length distributions, we can detect potential biases, such as an overrepresentation of shorter or longer sequences, which may impact model performance.

3.2.2 Hydrophobicity Analysis

Hydrophobicity affects ~~their~~ protein structural stability, folding, and interactions with cellular components~~ref?~~. The average hydrophobicity H of each sequence was computed using the Kyte-Doolittle scale (~~ref?~~):

$$H = \frac{1}{L} \sum_{i=1}^L H_i \quad (3.2)$$

where:

- H represents the average hydrophobicity of the sequence,
- L is the sequence length,
- H_i is the Kyte-Doolittle hydrophobicity score of the i^{th} amino acid.

By evaluating sequence length and hydrophobicity distributions, this analysis provides a foundational understanding of the dataset. These characteristics can help identify statistical outliers and reveal potential biases in the dataset.

I am not sure if this should be here. This sounds more like a discussion. Since I do not know if you are placing the results of this analysis, I cannot tell if this should be here, in the results or discussion. Keep it mind when writing the rest please.

3.3 Redundancy Reduction

Machine learning models trained on redundant sequences may become biased toward overrepresented patterns, reducing their generalizability. To mitigate this, redundancy reduction was applied to the dataset using MMseqs2[16].

The redundancy reduction process involved the following steps:

1. The protein sequences, originally stored in FASTA format, were converted into an MMseqs2 database format to facilitate clustering.
2. An initial clustering step was performed using a sequence identity threshold of 100% . This step ensured that exact duplicate sequences were removed, preventing redundant entries in the dataset. The following command was executed:

```
mmseqs easy-cluster file.fasta dataset tmp -min-seq-id 1.0 -cov-mode 0 -c 1.0
```

3. After removing exact duplicates, a second clustering step was applied using a sequence identity threshold of 30%. This step grouped similar sequences into clusters. The following command was used:

```
mmseqs easy-cluster file.fasta dataset tmp -min-seq-id 0.3 -cov-mode  
0 -c 1.0
```

The redundancy reduction workflow was implemented in the redundancy-reduction.py Python script.

3.4 Prediction of 3D Protein Folds

After applying redundancy reduction to the protein sequences, their three-dimensional (3D) structures were predicted using AlphaFold2. The structural confidence of these predicted protein folds is quantified using the per-residue *predicted Local Distance Difference Test* (*pLDDT*) score.

3.4.1 Extraction of Confidence Scores from AlphaFold2 Predictions

To evaluate the quality of the generated protein folds, a computational pipeline was developed to extract per-residue *pLDDT* scores from the predicted structures. This analysis was implemented using Python and the Biopython library that allows the parsing of Protein Data Bank (PDB) files. The processing pipeline consists of the following steps:

1. The program iterates through all PDB files generated by AlphaFold2, stored in the directory where each file corresponds to a single predicted protein structure.
2. The script employs the PDBParser module from Biopython to parse the structural information of each protein model. The *pLDDT* score is stored within the B-factor field of each atom in the PDB file. Since all atoms within a residue share the same *pLDDT* value, only one atom per residue is used to extract this confidence score.

What did you do with the results of this step? Do you have results? was it just exploratory? did you use the result to establish a threshold?

3.5 Preparation of Training and Test Datasets

Two different methods were applied to create the training and test datasets, in order to utilize redundancy reduction to minimize sequence similarity biases and mitigating effects of data imbalance.

3.5.1 Method 1: Similarity-Based Data Splitting

After applying redundancy reduction with a sequence identity threshold of 30%, sequences that exhibited more than 30% similarity to any other sequence were excluded from the training set. This approach ensures that the training dataset consists of unique sequences. Sequences with more than 30% sequence similarity were designated as the test set. Conversely, sequences with less than 30% similarity were assigned to the training set, forming a non-redundant subset for model learning.

3.5.2 Method 2: Random Splitting After Redundancy Reduction

In the second approach, sequences that had more than 30% similarity were first removed from the training set, similar to Method 1. However, instead of directly assigning similar sequences to the test set, the remaining non-redundant sequences were randomly split into training and test sets.

A stratified random split was performed using the `train_test_split` function from Scikit-Learn [version?](#), with a test size of 20% and a random state of 42 to ensure reproducibility:

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
```

3.6 Embedding Generation

In this study, protein embeddings were generated from both protein sequences and protein folds, utilizing protein language models: ProtT5 and ProST5.

For protein sequence based protein embedding generation, I employed the ProtT5 language model to generate embeddings for each protein sequence which encodes protein sequence features[17]. [how? which website? when was it accessed? etc.](#) [Methods have to be described in a way that someone can](#)

reproduce what you did. Since embeddings are only from outside sources, describe that you use a collab notebook, accessed (date), and follow the instructions by the authors. Same for the ProstT5

To incorporate structural information, embeddings were also generated using the ProstT5 language model from protein folds, which is trained to encode 3D structural features of proteins [18].

3.7 Dimensionality Reduction

To reduce the complexity of embedding vectors while preserving essential information, Principal Component Analysis (PCA) was employed. PCA transforms the high-dimensional embeddings into a lower-dimensional space by keeping maximum variance. [19]

Didem, you should add the file names performing the tasks as you did before. For models and their reproducibility, all different algorithms have specific hyperparameter and options that need to be declared.

3.8 Machine Learning Model Training and Evaluation

To ensure robust model validation, 5-fold cross-validation was applied, and Matthews Correlation Coefficient (MCC

Are you adding the formula somewhere? Then reference it here please.

) was used as a primary metric for model evaluation.

3.8.1 Model Selection

Several machine learning algorithms were selected for the classification tasks. The models used in this study include:

- Random Forest
- Logistic Regression
- Support Vector Machine (SVM)
- K-Nearest Neighbors (KNN)

Please add the reference to each of the type of models.

3.8.2 Additional Approach: Hierarchical Predictor

To address class imbalances and improve classification accuracy, a hierarchical predictor was implemented. This approach first uses a classifier to group exotoxins into broader type families based on their prevalence. A second-stage classifier is then applied within each group to refine the classification to specific exotoxin types. The architecture of this predictor is depicted in Figure 3.2 below.

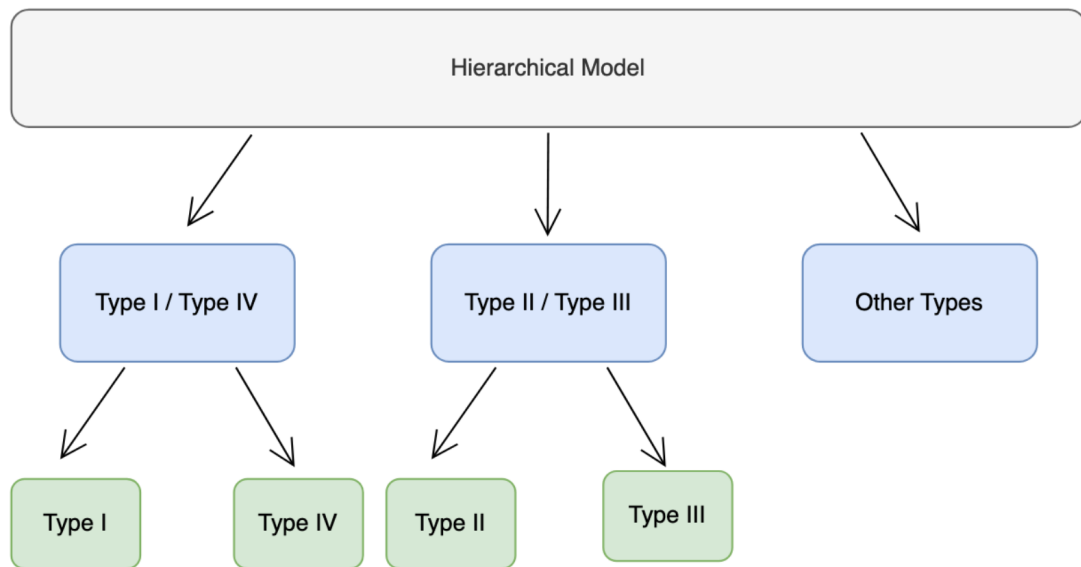


Figure 3.2: Architecture behind Hierarchical Models

Please extend the caption of the figure. Remember, a graph has to be able to stand on its own. First layer? second? third? what do they mean? Very short but to the point to understand the graph right away. For this assume expert knowledge. Since you are explaining it in the text in a more "non-experty" way, everyone can benefit of both approaches.

3.8.3 Additional Approach: BLAST-Based Predictor

As a baseline method, BLAST (Basic Local Alignment Search Tool) [ref?](#) was employed for sequence similarity analysis. To evaluate the predictive power of similarity-based classification, a custom BLAST database was constructed using the training sequences.

Each test protein sequence was queried against this database, and the best match was determined based on the lowest e-value, which indicates the statistical significance of the alignment. The predicted label of the matched sequence was then compared to the actual label in the test set to assess the accuracy of this approach.

Please make sure you are writing different from what we have in publication or in manuscript, or reference to a Kruger et al (in preparation). If not, our paper might be flag for plagiarism when your thesis is in the e-library of the University. I say this because it sound WAY similar to the manuscript Tanja and I are preparing...

3.8.4 Negative Control: Scrambled Protein Sequence Embeddings

To evaluate whether the machine learning models are based on meaningful biological patterns a negative control was used using scrambled protein sequences to generate protein embeddings.

For this study, protein sequences were randomly shuffled while preserving their amino acid composition. These scrambled sequences were then processed through the same embedding pipeline using ProtT5 and ProST5 to generate protein embeddings.

Dear Didem, I am missing here the deployment of the models (pickle?) which file has the info to use your models (you can reference your README.md in the github). When thinking about Material and Methods, think always: what information will I (you) need if I would start today with this project and I need to reproduce it to create something based on this.

Chapter 4

Results

This section presents two predictors, ExoType and ExoTarget, which differentiate bacterial exotoxin types and targets based on different inputs, including protein sequences and protein folds. We first describe the data exploration and preprocessing steps, then outline the feature analysis, and finally discuss the model training and evaluation results.

4.0.1 Data Exploration and Preprocessing

In this section, we describe the data distribution among exotoxin types and targets, and explain the preprocessing steps—including redundancy reduction—that prepare the dataset for modeling.

Dataset Overview

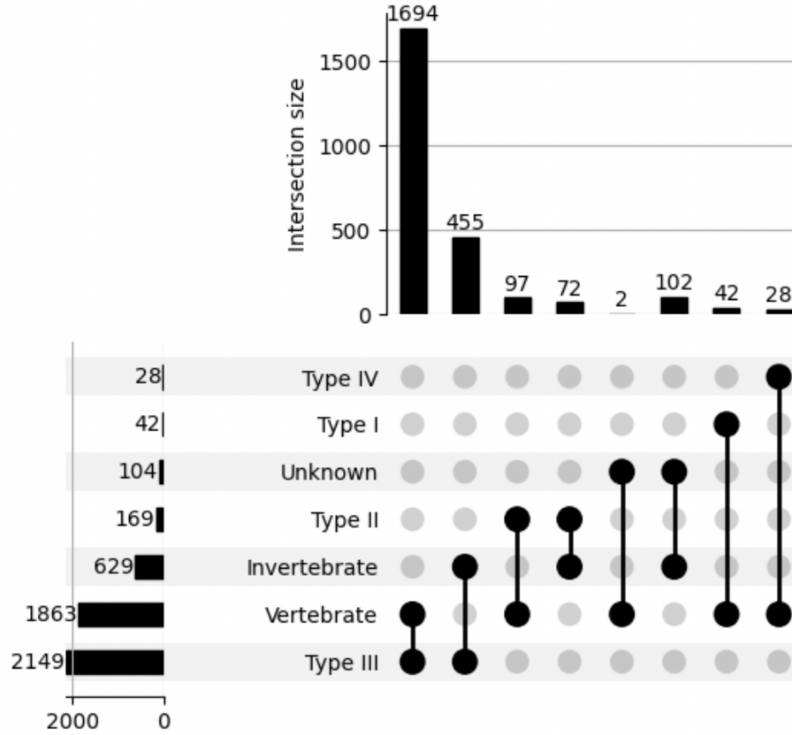


Figure 4.1: Upset plot illustrating the distribution of exotoxin types and their host targets. This plot is used to visualize the intersections between categories and to highlight class imbalances in the dataset. The upper bar chart shows the intersection sizes, with the tallest bar (approximately 1,694) indicating the most common category intersection. The lower matrix, composed of black-filled circles on a gray background, maps each exotoxin type (Type I, Type II, Type III, Type IV, Unknown) and host target (Vertebrate, Invertebrate) to its intersection, with the horizontal bars indicating the total sequence counts (e.g., Type III with 2,149 sequences, Vertebrate-targeting with 1,863 sequences, and Invertebrate-targeting with 629 sequences).

Figure 4.1 summarizes the distribution of exotoxin types and targets within the dataset. Notably, the dataset is heavily skewed toward Type III exotoxins (2,149 sequences), which indicates a significant class imbalance that could affect model performance. In contrast, exotoxins categorized as “Unknown” comprise only 104 samples. Additionally, target classification shows that

bacterial exotoxins targeting vertebrates (1,863 sequences) are three times more abundant than those targeting invertebrates (629 sequences). These observations highlight the need for mitigating data imbalance which will be observed in following model training step.

4.0.2 Redundancy Reduction

To ensure robustness and generalizability, redundancy reduction was applied to the exotoxin dataset. First, using MMseqs2 with a 100% similarity threshold, duplicate sequences were removed, reducing the total count from 2,497 to 2,492. Next, applying a 30% similarity threshold further reduced the count from 2,492 to 1,068. This reduction minimizes sequence similarity, allowing us to focus on the unique characteristics and functional diversity of exotoxins.

As shown in Table 4.1, after eliminating redundant sequences, the over-representation of Type III exotoxins remains even more pronounced, which could increase prediction bias. Such imbalances highlight the need for techniques like oversampling or weighted loss functions during model training.

Therefore, generation of these non-redundant data allows further analysis of exotoxin properties such as sequence length and hydrophobicity.

Table 4.1: Summary of the dataset distribution based on exotoxin types and their biological targets before and after redundancy reduction.

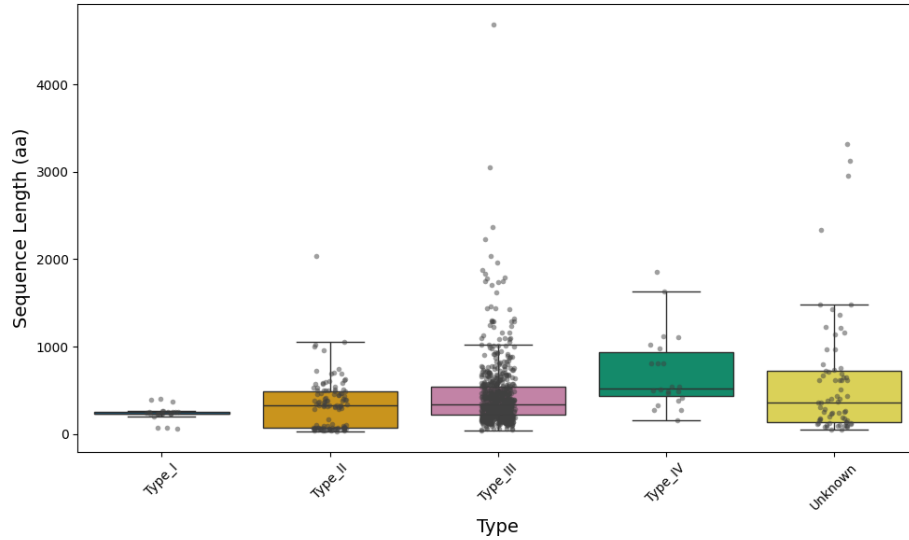
(a)			(b)		
Exotoxin Type	Original	Reduced	Exotoxin Target	Original	Reduced
Type I	42	29	Vertebrates	1863	614
Type II	169	145	Invertebrates	629	234
Type III	2149	852			
Type IV	28	25			
Other Types	104	93			

4.0.3 Feature Analysis of Exotoxins

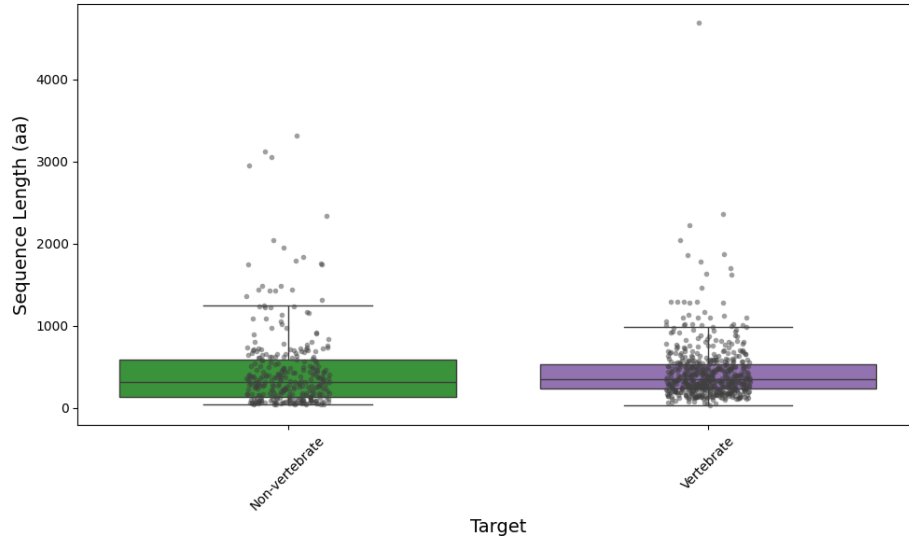
In this section, we examine key sequence features such as sequence length and hydrophobicity to assess potential biases that could influence predictive modeling.

Sequence Length Analysis

Following redundancy reduction, a descriptive analysis of sequence lengths was conducted using boxplots with nonredundant dataset. Figure 4.2a shows the distribution of sequence lengths (measured in amino acids) for each exotoxin type, while Figure 4.2b displays the distribution by target. These plots are used to identify differences in sequence length among classes.



(a) **Sequence length distribution for different bacterial exotoxin types.**(measured in amino acids). The x-axis represents the exotoxin types and each toxin type is represented by a distinct colors: Type I (blue), Type II (orange), Type III (pink), Type IV (green), and Unknown (yellow). The y-axis shows sequence length in amino acids, ranging from 0 to approximately 3000 except three outliers.



(b) **Sequence length distribution for different bacterial exotoxin targets.**(measured in amino acids). The x-axis represents the exotoxin targets, and each toxin type is represented by a distinct color: Vertebrates (dark green) and Non-vertebrates (purple). The y-axis shows sequence length in amino acids, ranging from 0 to approximately 2500 except five outliers.

Figure 4.2: Descriptive Analysis of Bacterial Exotoxin Feature: Sequence Length

The statistical analysis via generalized linear models revealed that Type III, Type IV, and Unknown exotoxins have significantly longer sequences (with coefficients of 209.93, 466.13, and 374.21, respectively; all with p-values less than 0.016). In contrast, Type II and vertebrate-targeting exotoxins show no significant length difference. The residuals plot in Figure 4.3 confirms that sequence length alone does not fully capture the variance, suggesting additional factors may be involved.

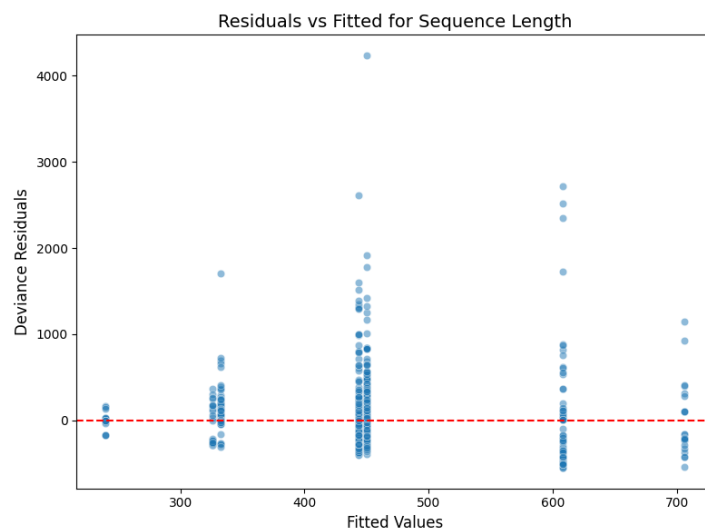
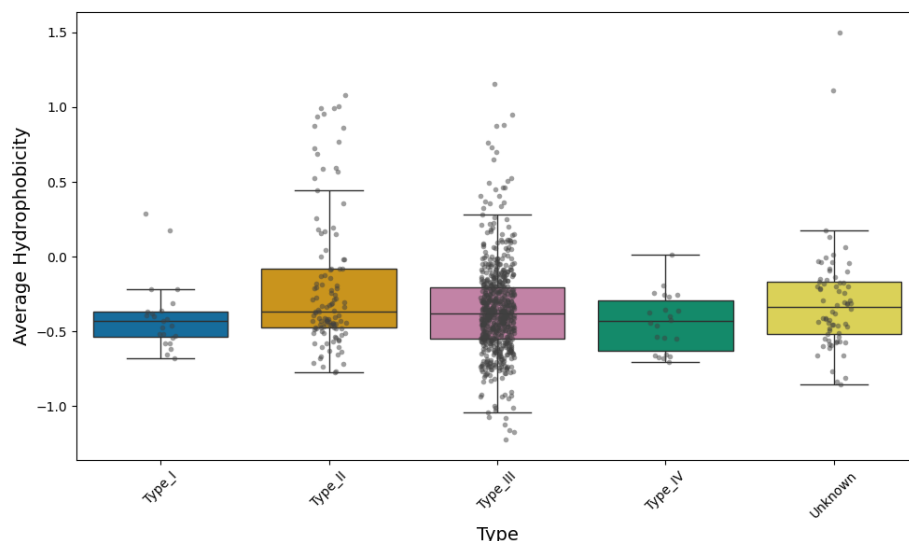


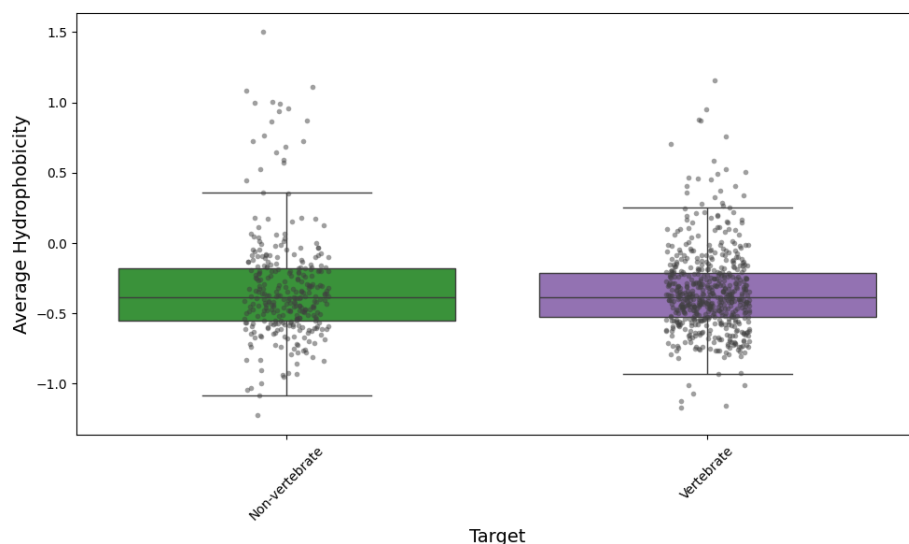
Figure 4.3: Deviance residuals vs. fitted values for the GLM on sequence length. This plot is used to assess model fit and shows that large residuals indicate unexplained variance in sequence length across exotoxin types.

Hydrophobicity Feature Analysis

The average hydrophobicity was analyzed (values calculated per residue) across different exotoxin types and targets. Figure 4.4a shows the hydrophobicity distribution by type, while Figure 4.4b presents the data by target. These boxplots help to determine whether hydrophobicity varies systematically among the classes.



(a) **Hydrophobicity distribution for different bacterial exotoxin types.**(values calculated per residue). The x-axis represents the exotoxin types and each toxin type is represented by a distinct color: Type I (blue), Type II (orange), Type III (pink), Type IV (green), and Unknown (yellow). The y-axis shows the average hydrophobicity values, ranging from approximately -1.5 to 1.0. Each box plot displays the median hydrophobicity, interquartile range (IQR), and whiskers extending up to 1.5 times the IQR. Individual outliers are marked as points outside the whiskers. The hydrophobicity values vary slightly across toxin types, with Type II showing a wider spread compared to others, while Type I and Type IV are more compact.



(b) **Hydrophobicity distribution for different bacterial exotoxin targets.**(values calculated per residue). The x-axis represents the exotoxin targets, and each toxin target is represented by a distinct color: Vertebrates (dark green) and Non-vertebrates (purple). The y-axis shows the average hydrophobicity values, ranging from approximately -1.5 to 1.0. Each box plot summarizes the data, showing the median hydrophobicity, interquartile range (IQR), and whiskers extending up to 1.5 times the IQR, with outliers plotted as individual points. Both vertebrate and non-vertebrate targets show a similar distribution of hydrophobicity values.

Generalized linear models indicate that only Type II toxins have significantly higher hydrophobicity (coefficient = 0.2075, $p = 0.007$), while other types and vertebrate targets do not show significant differences. The residuals plot in Figure 4.5 further suggests that toxin type alone does not provide enough information to predict hydrophobicity.

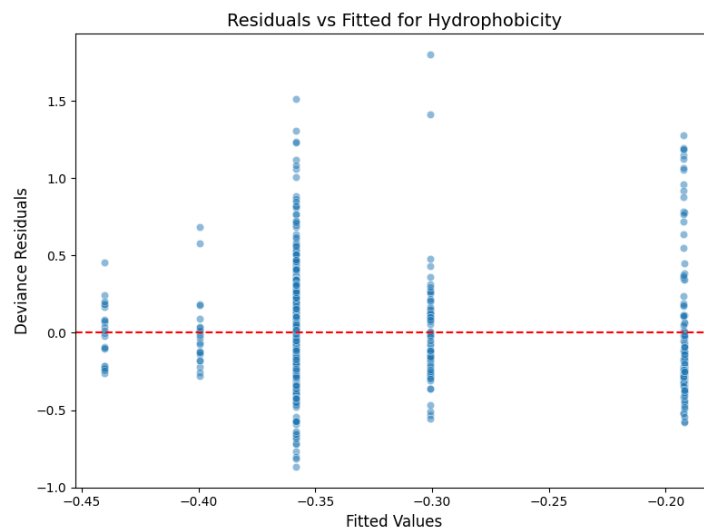


Figure 4.5: Deviance residuals vs. fitted values for the GLM on hydrophobicity. The scattered residuals suggest that toxin type alone does not strongly predict hydrophobicity, indicating the need for additional explanatory variables.

4.0.4 Quality of Protein Folds Based On pLDDT Scores

Figure 4.6 displays the distribution of pLDDT scores (obtained from the B-factor column) for predicted protein folds. This figure is used to evaluate the confidence of our protein structure predictions; higher scores indicate greater model confidence, while lower scores suggest regions that may require further analysis.

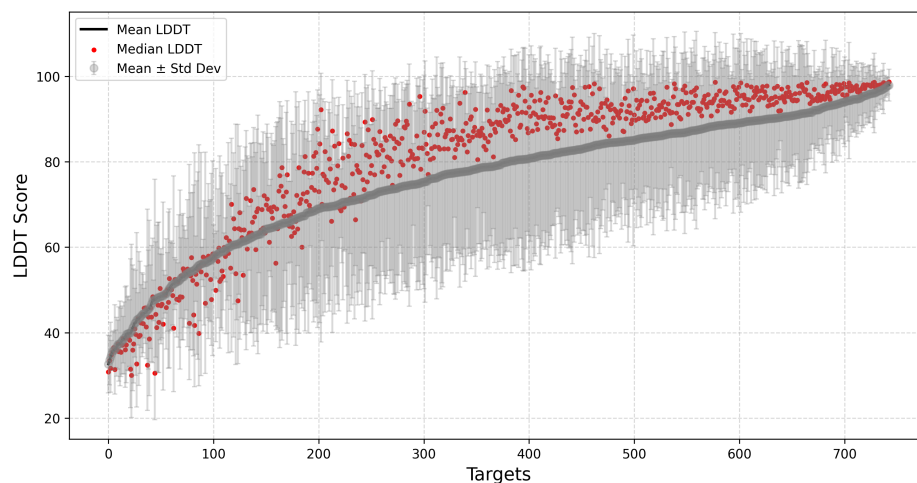
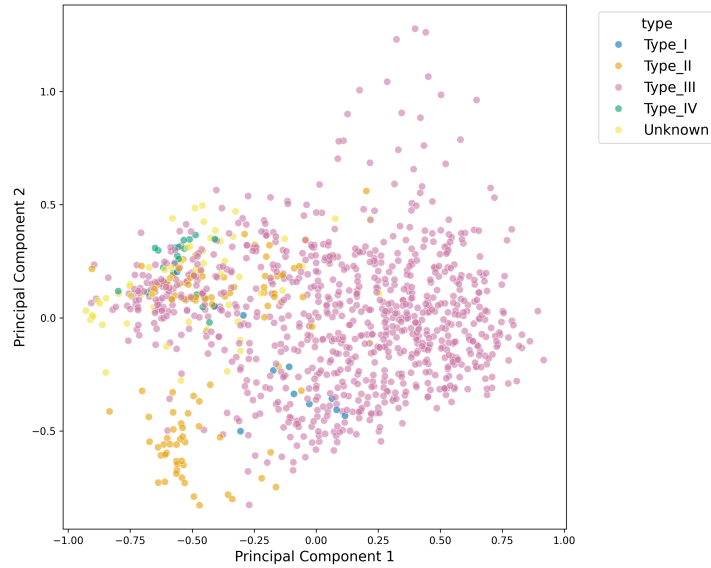


Figure 4.6: LDDT Score Distribution Across Targets. The black line represents the mean LDDT score for each target, while the gray error bars indicate the standard deviation. Red points denote the median LDDT score for each target. This visualization highlights the distribution of model confidence across different protein structures. Low-confidence predictions correspond to $\text{LDDT} < 50$, high confidence regions correspond to $\text{LDDT} > 70$.

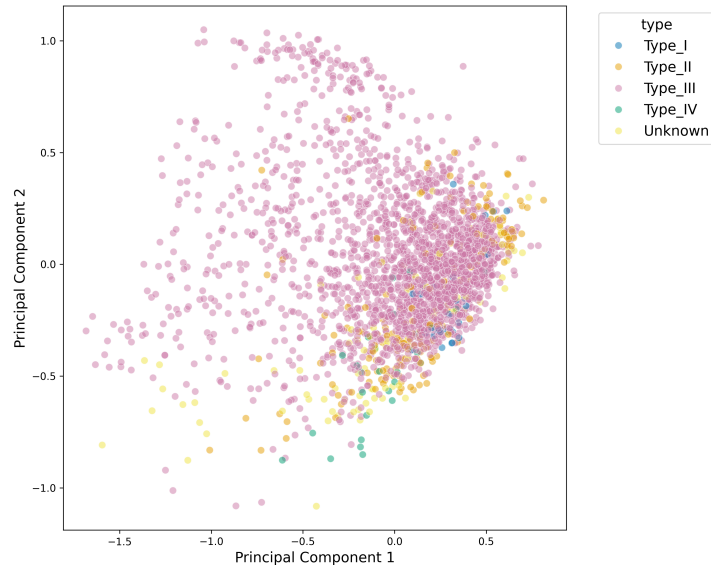
4.0.5 Dimensionality Reduction Techniques

Principal Component Analysis (PCA) was applied to both standard and scrambled datasets to assess the clustering of sequences based on their features. These analyses help us understand whether the learned embeddings capture biologically meaningful differences.

In Figure 4.7a, the PCA projection for exotoxin types shows that Type II and Type IV tend to cluster distinctly, whereas Type III is more dispersed with some overlap among types. In contrast, the scrambled PCA in Figure 4.7b shows no visible structure, confirming that the original clustering is biologically meaningful.



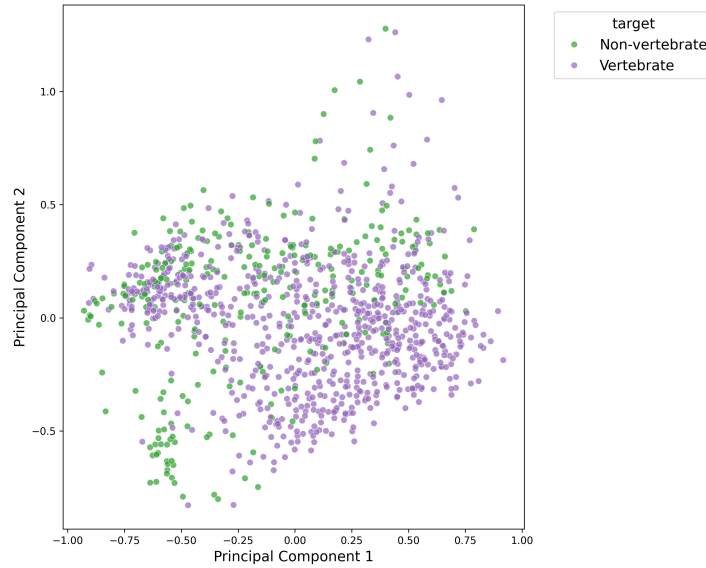
(a) **Principal Component Analysis (PCA) projection of bacterial exotoxin types** The x-axis represents Principal Component 1, and the y-axis represents Principal Component 2, both ranging from approximately -1.0 to 1.0. Each exotoxin type is color-coded: Type I (blue), Type II (purple), Type III (pink), Type IV (green), and Unknown (yellow). Clusters suggest some overlap but distinct separations for specific types.



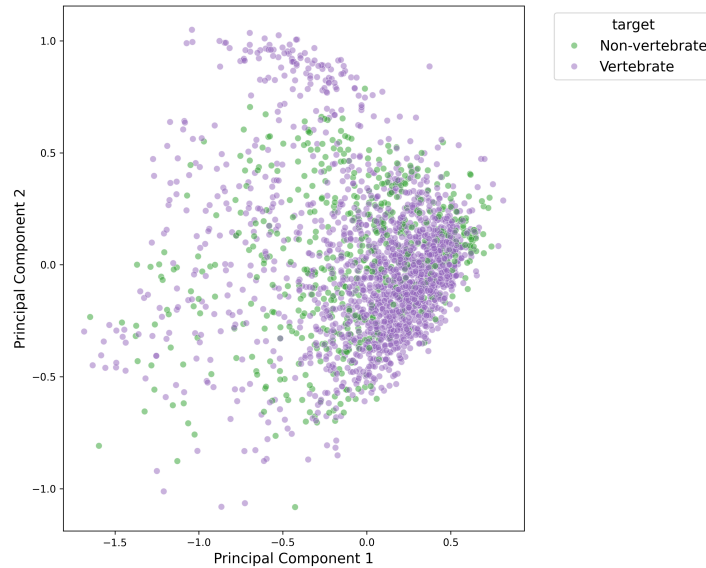
(b) **Scrambled PCA projection of bacterial exotoxin types.** The x-axis represents Principal Component 1, and the y-axis represents Principal Component 2, with a wider range of values between -1.5 and 1.0. The same color coding applies: Type I (blue), Type II (purple), Type III (pink), Type IV (green), and Unknown (yellow). The scrambling reduces separation among the clusters, validating the effect of shuffling of protein sequences.

Figure 4.7: Principal Component Analysis Based On Exotoxin Types For Dimensionality Reduction

Similarly, Figure 4.8a and Figure 4.8b show the PCA projections for exotoxin targets, confirming that vertebrate and invertebrate sequences exhibit biologically meaningful clustering.



(a) Principal Component Analysis (PCA) projection of bacterial exotoxin targets. The x-axis represents Principal Component 1, and the y-axis represents Principal Component 2, both ranging from approximately -1.0 to 1.0. Targets are color-coded: Vertebrates (purple) and Non-vertebrates (dark green). Vertebrate clusters are more compact, while non-vertebrate clusters show broader variability.



(b) Scrambled PCA projection of bacterial exotoxin targets. The x-axis represents Principal Component 1, and the y-axis represents Principal Component 2, with a wider range of values (-1.5 to 1.0). Targets are color-coded as Vertebrates (orange) and Non-vertebrates (blue). The scrambling reduces cluster compactness, demonstrating the loss of structure in the data.

Figure 4.8: Principal Component Analysis Based On Exotoxin Targets For Dimensionality Reduction

4.0.6 ExoType Predictor

This section describes the development of the ExoType predictor, which classifies exotoxins into five categories: Type I, Type II, Type III, Type IV, and Unknown. The following subsections detail the training and evaluation of the model.

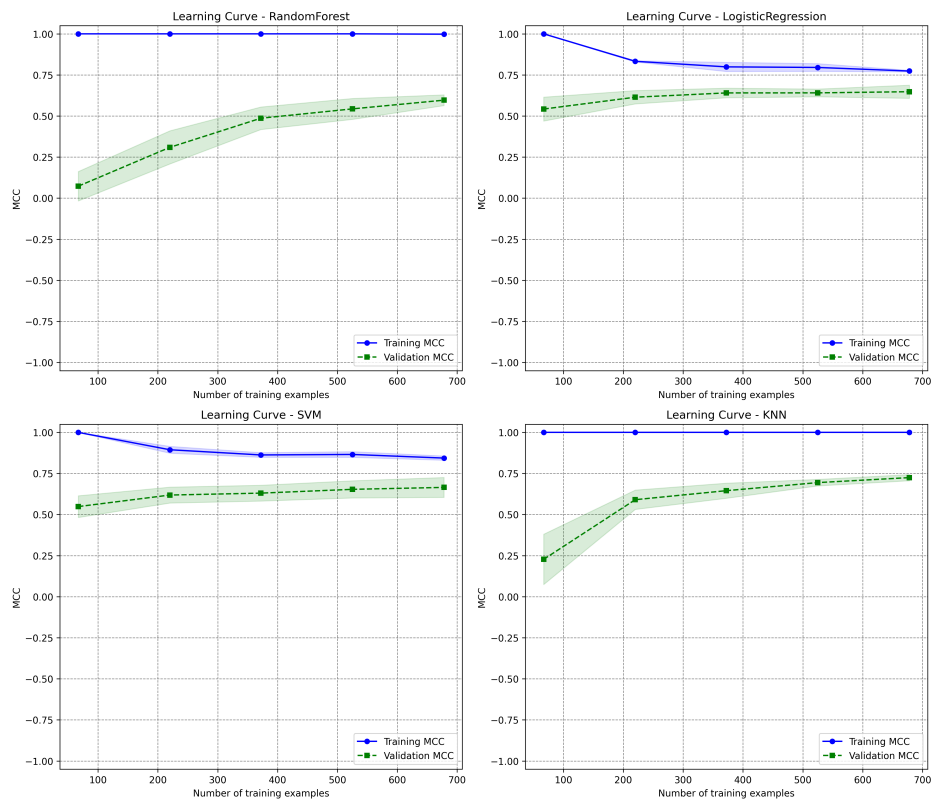
4.0.7 Model Training

To ensure robust evaluation and mitigate class imbalance, we employed 5-fold cross-validation with `class_weight=balanced` and GridSearchCV for hyperparameter tuning. Initially, the preprocessed dataset was split into an 80%-20% training-test split to evaluate generalization on unique sequences. However, due to the imbalanced distribution of exotoxin classes, some models (e.g., KNN and Logistic Regression) exhibited overfitting, as indicated by a flat training curve in Figure 4.9a.

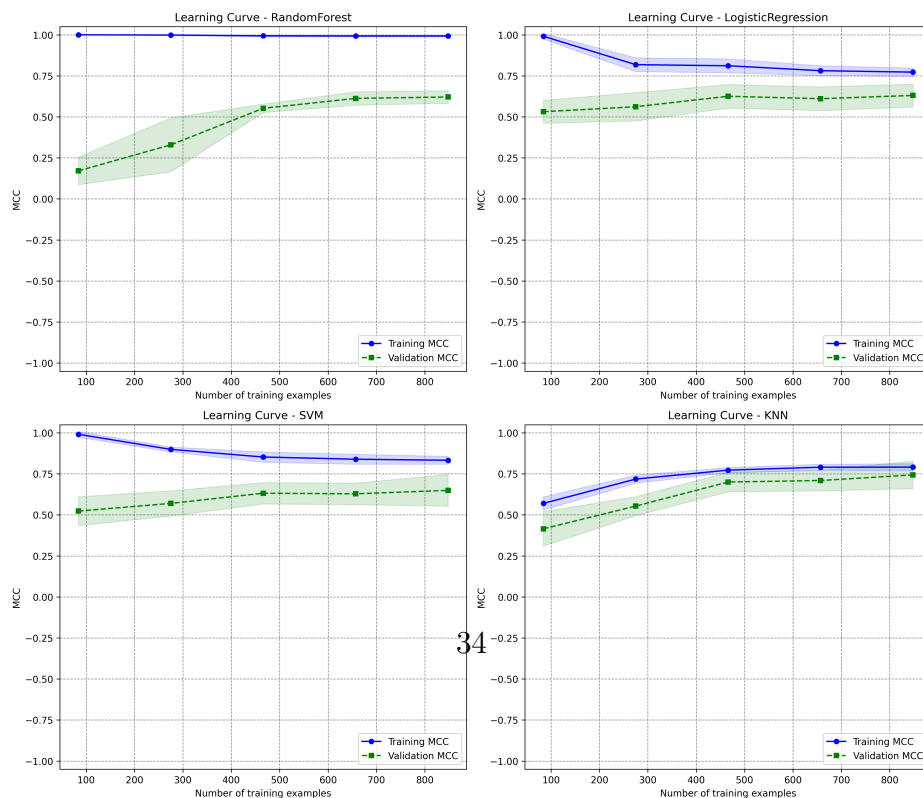
In a second approach, the entire non-redundant dataset was used for training, and the previously eliminated redundant sequences were used as the test set. As shown in Figure 4.9b, this approach improved the performance of KNN and Logistic Regression, while Random Forest and SVM maintained strong performance with smoother learning curves.

Learning Curves for Protein Sequence-Based Embeddings

While the first approach displays a more traditional generalization test, the second approach benefits from a richer training set at the cost of testing on potentially redundant data. Random Forest and SVM performed well in both cases, but the first approach led to greater variability in KNN and Logistic Regression, making them less reliable for handling unseen data.



(a) This graph is used to assess model generalization when the dataset is split into 80% training and 20% test sets.



(b) This plot shows model performance when using the full non-redundant dataset for training and the redundant data as the test set.

Learning Curves for Protein Fold-Based Embeddings

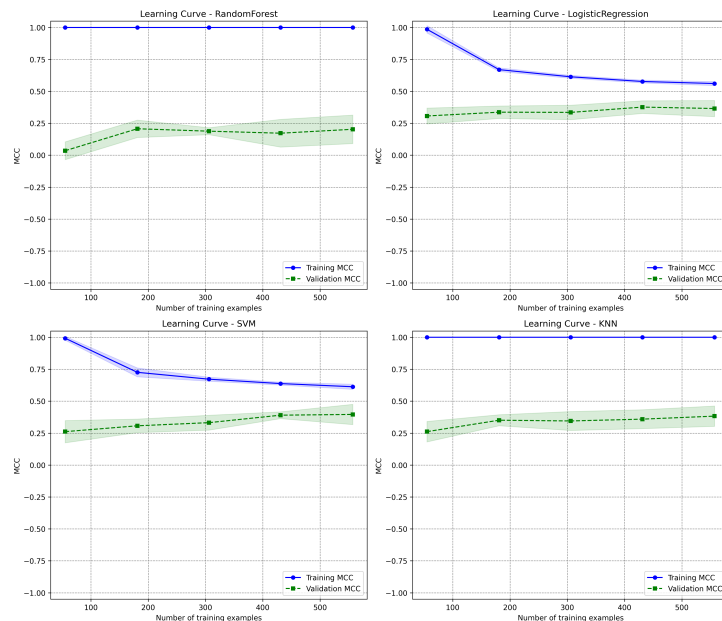


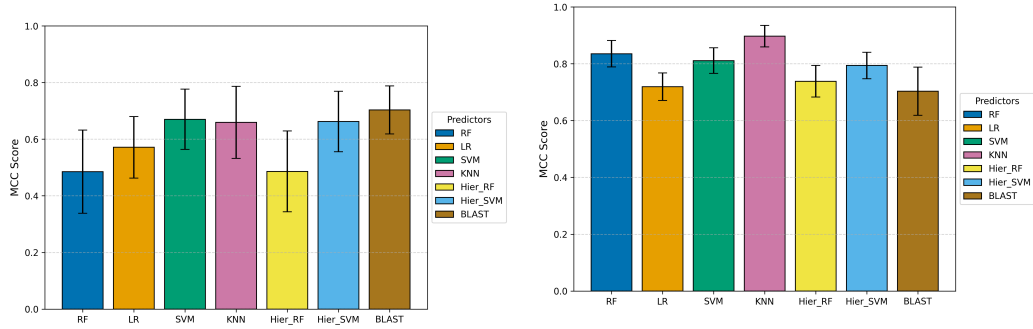
Figure 4.10: MCC Bar Plot for Protein Fold-Based Embeddings

4.0.8 Performance Evaluation

After testing different data-splitting methods, a hierarchical predictor was built to mitigate class imbalance. We evaluated model performance using MCC, ROC, and Precision-Recall curves.

Figure 4.11 shows MCC bar plots for protein sequence-based embeddings. The top panel (split test) indicates that SVM (0.67) and KNN (0.68) outperform other models, while the bottom panel (import test) shows improved performance, with SVM and KNN exceeding 0.80. The hierarchical models further improve performance, particularly Hierarchical SVM.

MCC Bar Plots for Protein Sequence-Based Embeddings



(a) This graph is used to compare model performance using an 80%-20% split.

(b) This graph compares performance when using the full non-redundant dataset for training.

Figure 4.11: MCC Bar Plots for Protein Sequence-Based Embeddings. The x-axis represents different classification models: Random Forest (RF, blue), Logistic Regression (LR, orange), Support Vector Machine (SVM, green), K-Nearest Neighbors (KNN, pink), Hierarchical Random Forest (Hier_RF, yellow), Hierarchical Support Vector Machine (Hier_SVM, light blue), and BLAST predictor (brown). The y-axis indicates the Matthews Correlation Coefficient (MCC) scores, with higher values reflecting better classification performance. Error bars represent the standard deviation across cross-validation folds. (a) First MCC Plot corresponds to the first approach, where the dataset was split into 80% training and 20% test set, while (b) Second MCC Plot represents the second approach, where the entire non-redundant dataset was used for training, and previously removed redundant data was used as the test set. Models trained using the second approach generally show higher MCC scores, demonstrating the benefit of increased training data. The hierarchical models (Hier_RF, Hier_SVM) exhibit improved MCC scores, highlighting their ability to mitigate class imbalance.

The graph on the top in Figure 4.11, SVM (0.67) and KNN (0.68) outperform other models, while Random Forest (0.55) and BLAST (0.60) show moderate performance. Hierarchical SVM (0.62) improves over standard SVM, indicating better handling of class imbalance. Logistic Regression and Hierarchical Random Forest perform worse, with MCC values below 0.50.

The graph at the bottom in Figure 4.11, SVM and KNN exceed 0.80, while

Hierarchical SVM reaches 0.78, reinforcing its robustness. Random Forest (0.75) and BLAST (0.72) improve notably, but Hierarchical Random Forest (0.65) still lags behind. The second approach results in higher MCC scores across models, showing the advantage of a larger training set. Hierarchical models consistently perform better, particularly Hierarchical SVM, which balances class distributions effectively.

MCC Bar Plots for Protein Protein Fold-Based Embeddings

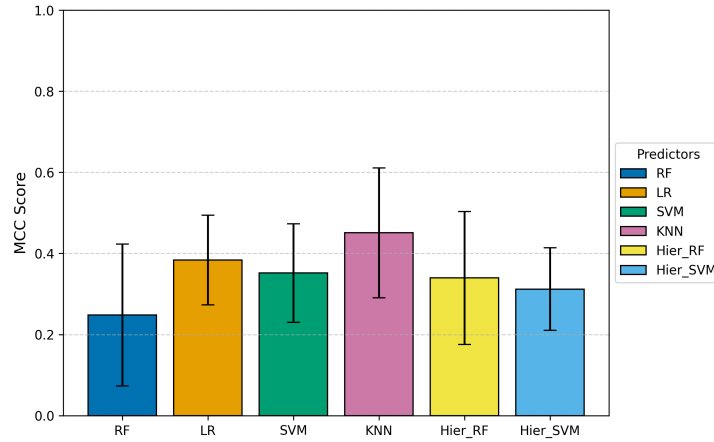
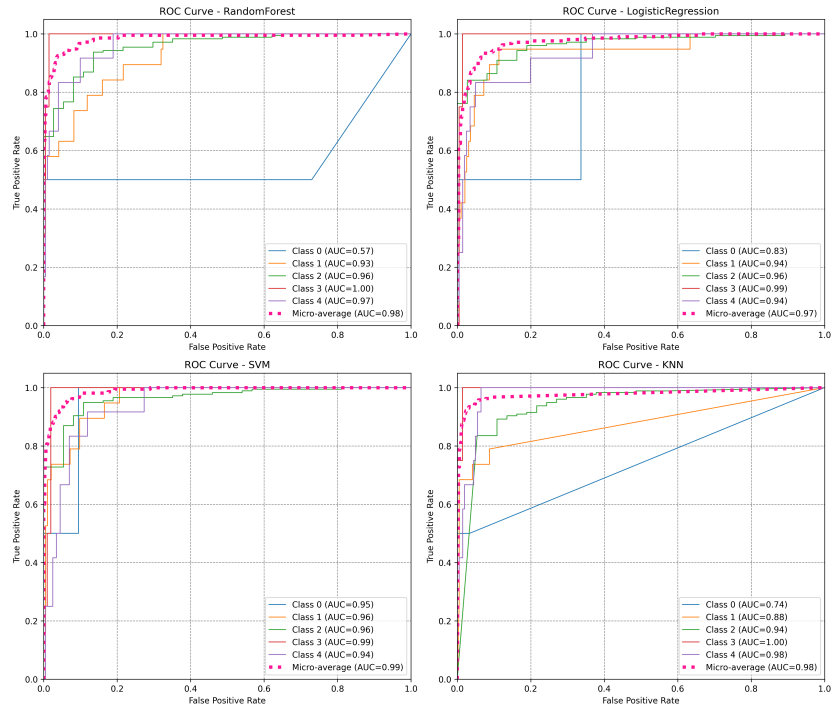


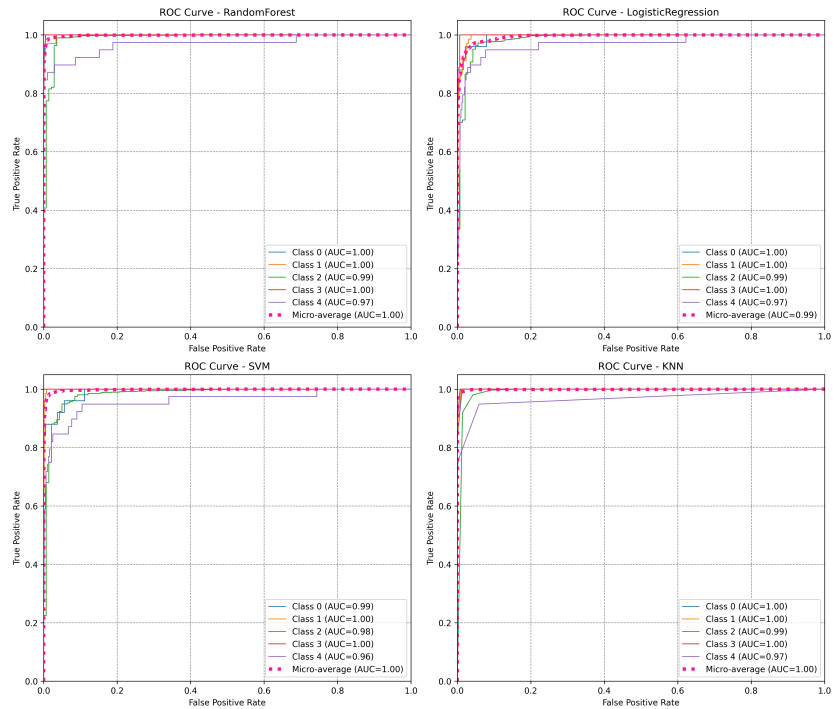
Figure 4.12: MCC Bar Plot for Protein Fold-Based Embeddings

ROC Curves for Protein Sequence-Based Embeddings

Figures 4.21 and 4.23 show ROC and Precision-Recall curves, respectively, for protein sequence-based embeddings, while Figures 4.22 and 4.24 present the corresponding curves for protein fold-based embeddings. These curves are used to evaluate the trade-offs between sensitivity and specificity (ROC) and precision versus recall, confirming that the hierarchical models achieve a balanced performance.



(a) This curve is used to evaluate the sensitivity and specificity for a split test.



(b) This curve is used to assess model performance with increased testing data.

Figure 4.13

ROC Curves for Protein Protein Fold-Based Embeddings

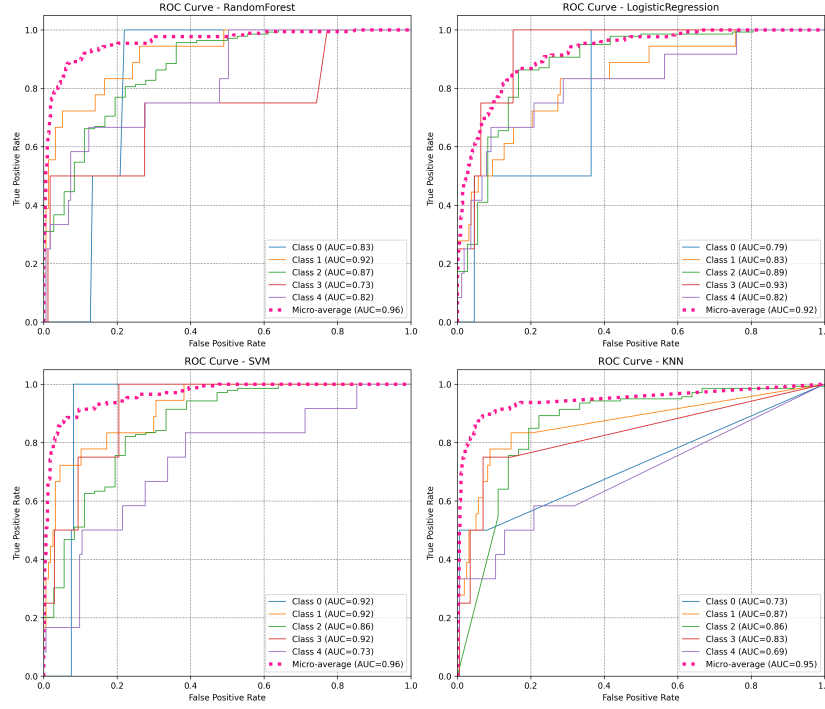


Figure 4.14: ROC Curve for Protein Fold-Based Embeddings. This plot is used to evaluate the performance of models based on protein fold features.

Figure 4.13 presents the ROC curves for multiple classification models trained on sequence-based features, evaluating their performance under different testing conditions. The results indicate that Random Forest and SVM consistently achieve high AUC values across all classes, demonstrating strong generalization. Logistic Regression also performs well but exhibits some differences in sensitivity for certain classes. In contrast, KNN shows inconsistent performance, suggesting that the model struggles with the given features.

Figure 4.14 illustrates the ROC curves for models trained on protein fold-based embeddings. While the overall performance remains high, the micro-average AUC values are slightly lower compared to Figure 4.13, indicating increased classification complexity. Notably, some classes experience a decline in separability, particularly in the Logistic Regression and SVM models, highlighting potential challenges in utilizing fold-based embeddings, proba-

bly related to fold quality. These comparisons suggest that while protein fold-based embeddings capture structural information, they may introduce noise in classification tasks compared to sequence-based features.

Precision-Recall Curves for Protein Protein Sequence-Based Embeddings

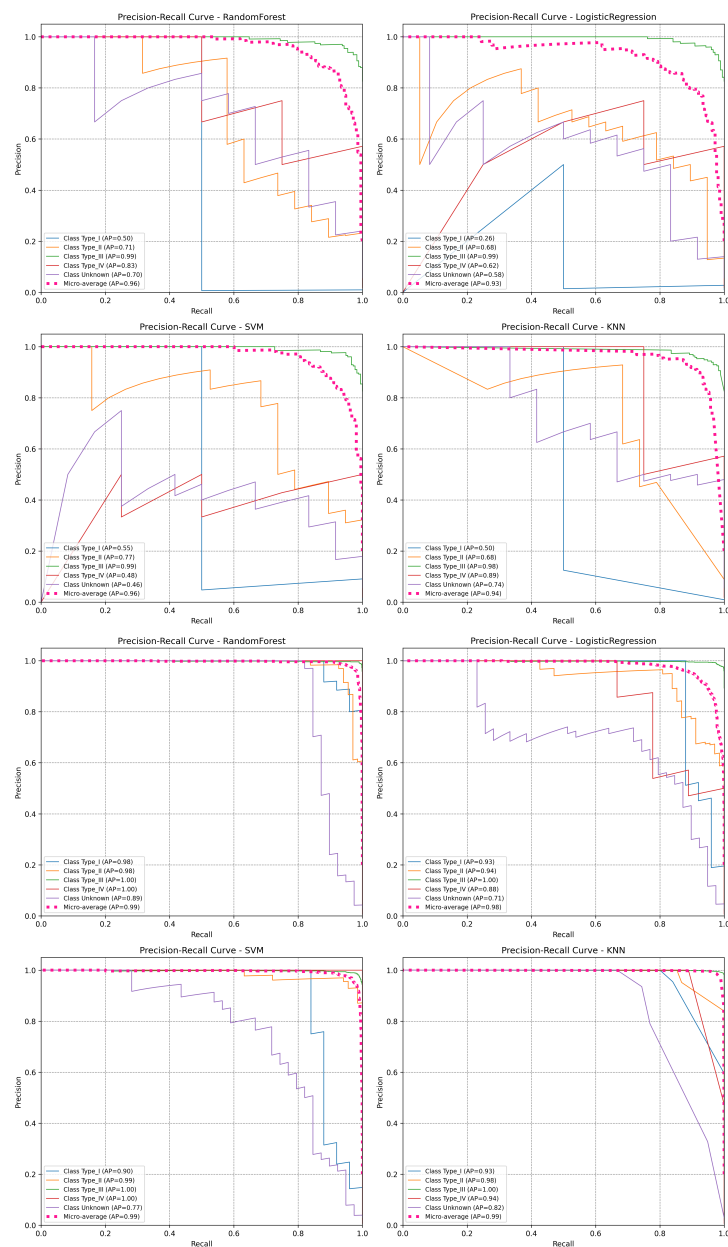


Figure 4.15: Precision-Recall Curves for Protein Sequence-Based Embeddings. These curves are used to assess the balance between precision and recall across different models.

Precision-Recall Curves for Protein Protein Fold-Based Embeddings

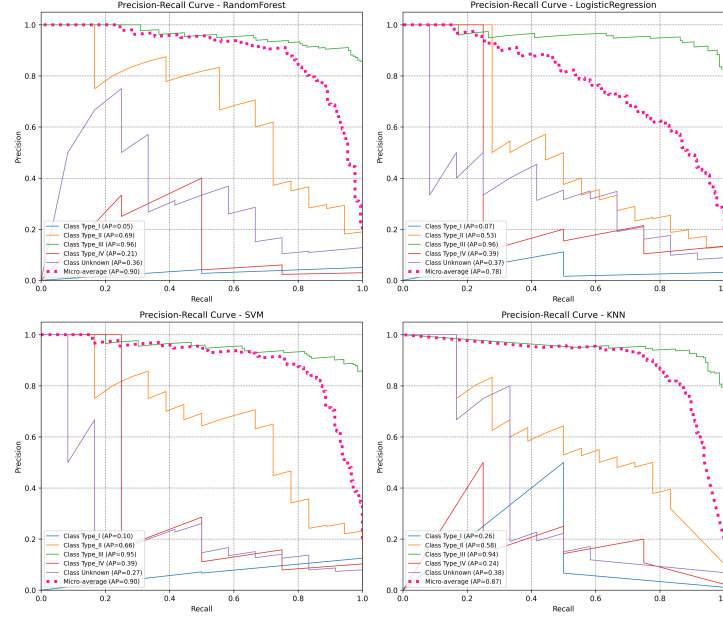


Figure 4.16: Precision-Recall Curve for Protein Fold-Based Embeddings. This plot is used to evaluate the precision-recall trade-off for models using protein fold-based features.

In Figure 4.15, models trained on sequence-based embeddings generally achieve higher micro-average precision-recall scores, indicating better classification performance. Random Forest and SVM exhibit strong PR curves with more stable precision across different recall values, suggesting that these models can balance false positives and false negatives. Logistic Regression has competitive performance, while KNN has poor one.

In Figure 4.16, models trained on fold-based embeddings show a decrease in precision-recall performance compared to Figure 4.15. The PR curves display more variability, particularly for Logistic Regression and KNN, indicating that fold-based embeddings introduce more challenges during the classification task. Lastly, Random Forest and SVM still perform relatively well.

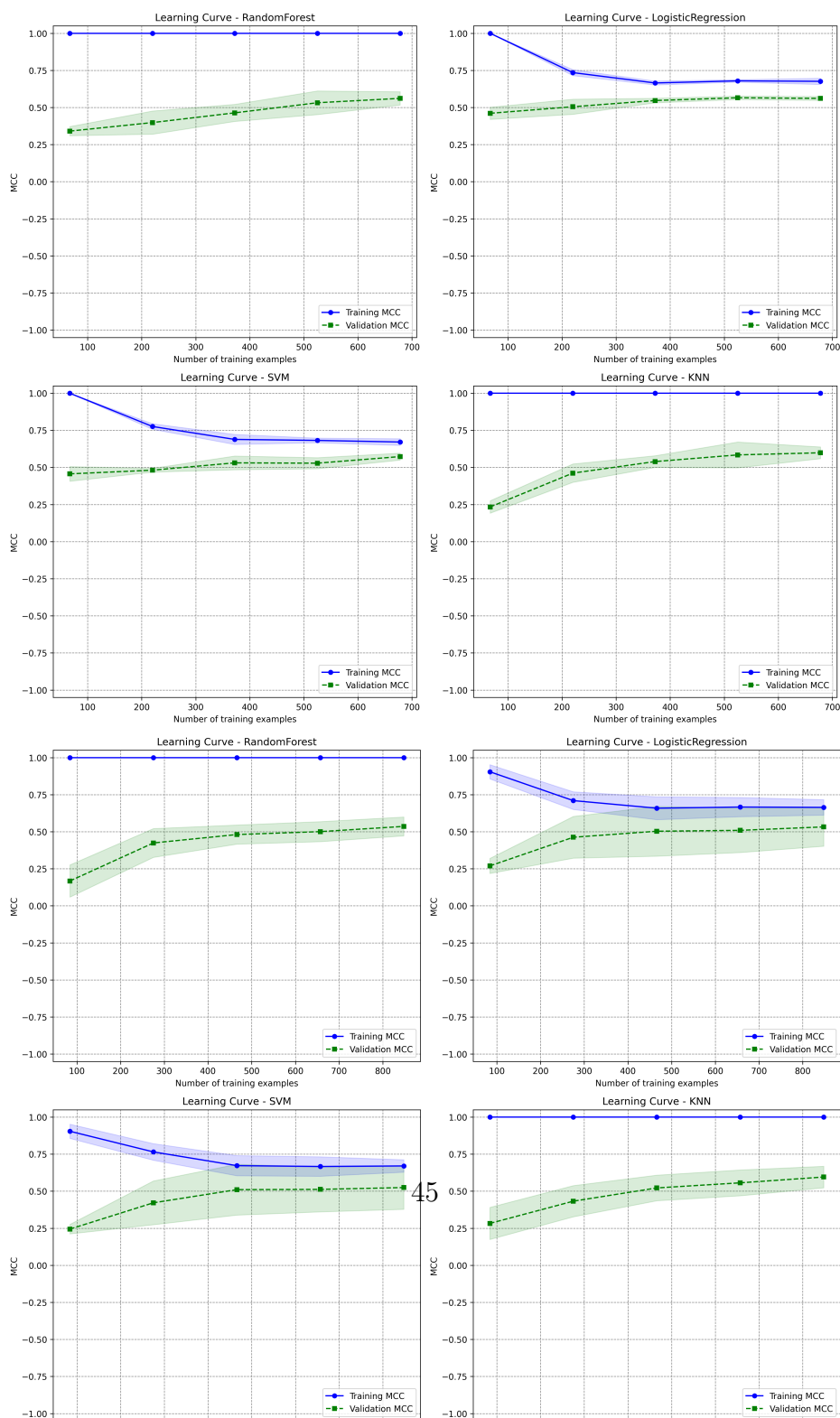
Overall, the comparison between Figures 4.15 and 4.16, sequence-based

embeddings lead to better classification performance, while fold-based embeddings introduce challenges that makes classification more difficult. Random Forest and SVM consistently outperform other classifiers, while KNN remains the least effective across both feature representations.

4.0.9 ExoTarget Predictor

4.0.10 Model Training

Learning Curves for Protein Sequence-Based Embeddings



Learning Curves for Protein Protein Fold-Based Embeddings

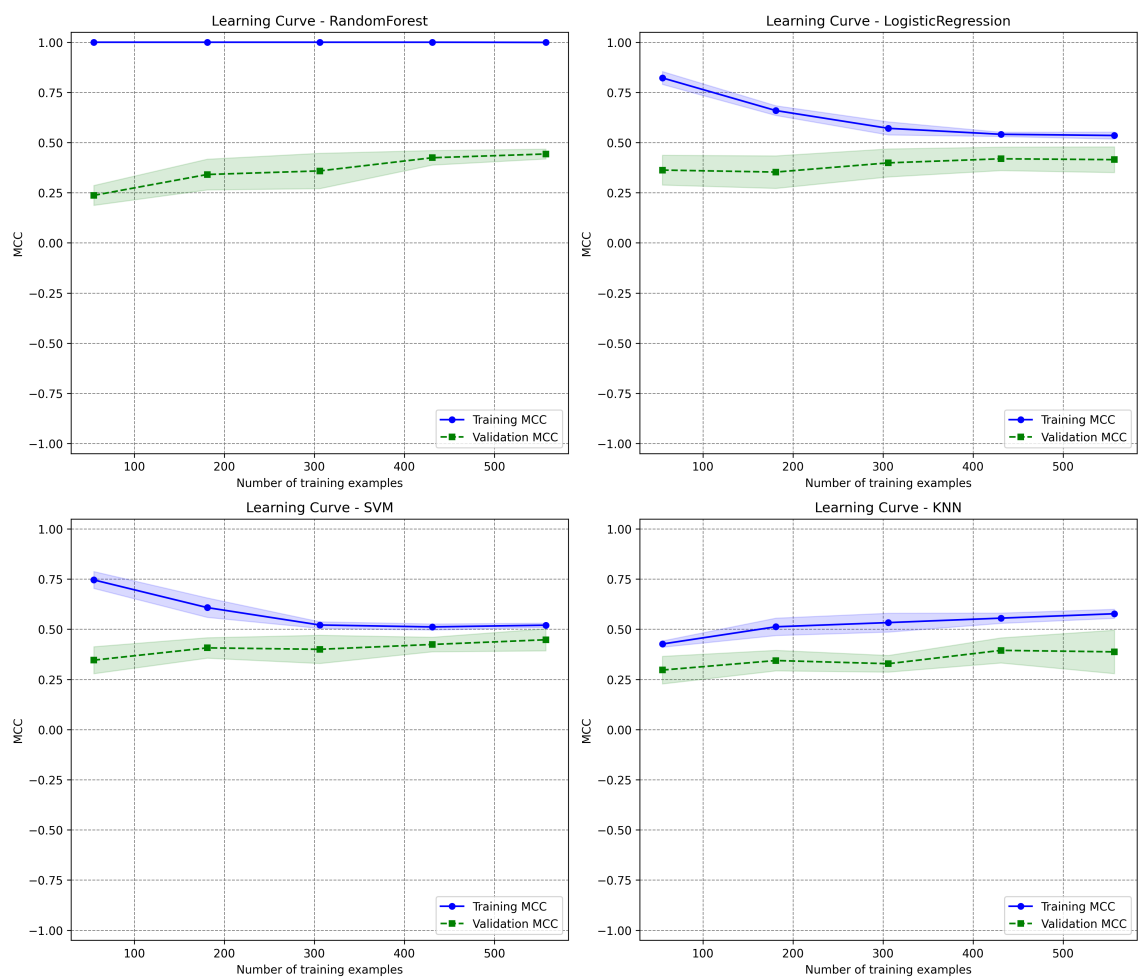


Figure 4.18: Learning Curve for Protein Fold-Based Embeddings

4.0.11 Performance Evaluation

MCC Bar Plots for Protein Sequence-Based Embeddings

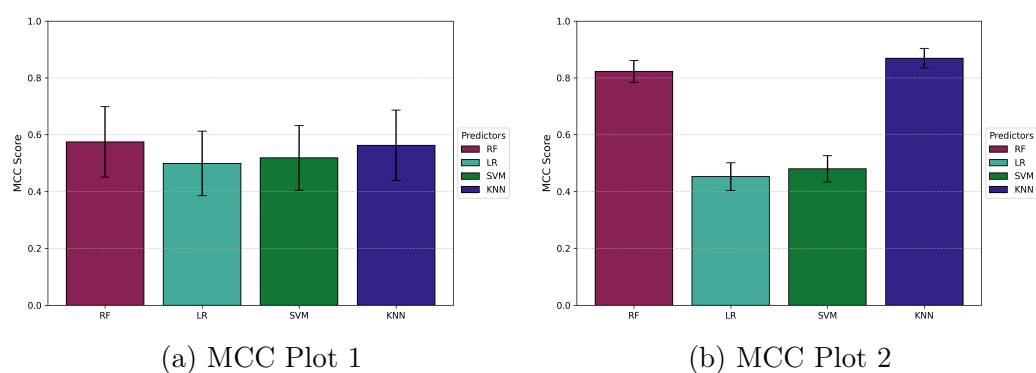


Figure 4.19: MCC Bar Plots for Protein Sequence-Based Embeddings

MCC Bar Plots for Protein Protein Fold-Based Embeddings

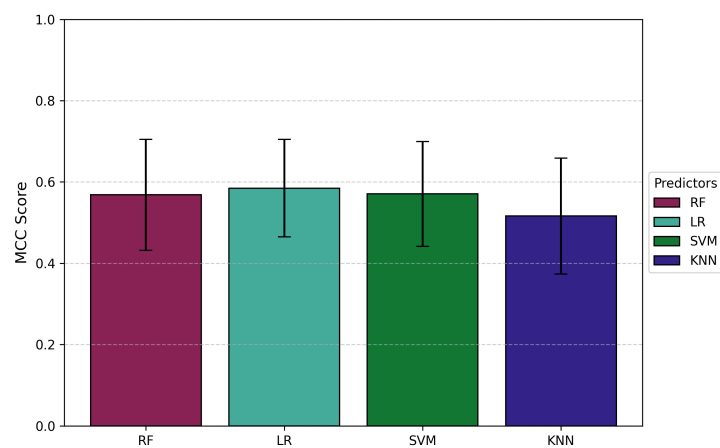


Figure 4.20: MCC Bar Plot for Protein Fold-Based Embeddings

ROC Curves for Protein Sequence-Based Embeddings

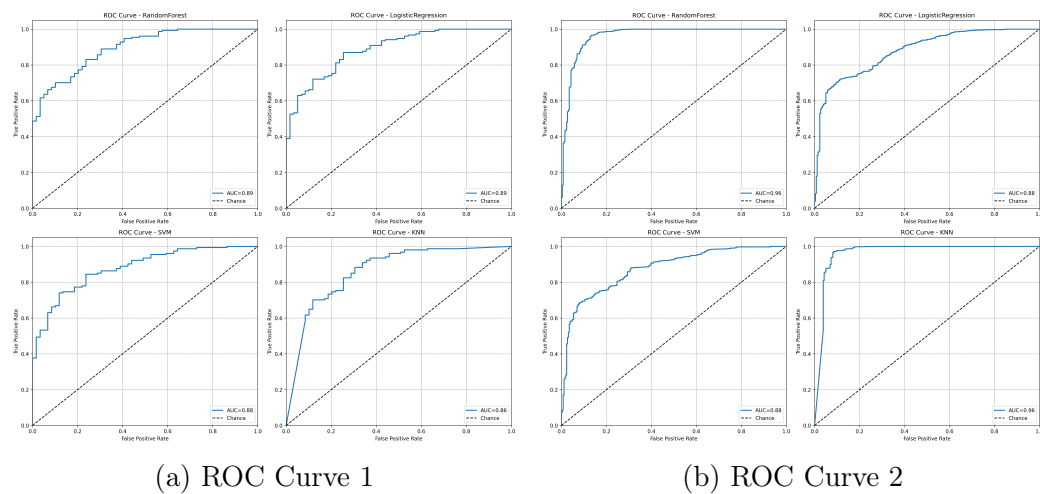


Figure 4.21: ROC Curves for Protein Sequence-Based Embeddings

ROC Curves for Protein Protein Fold-Based Embeddings

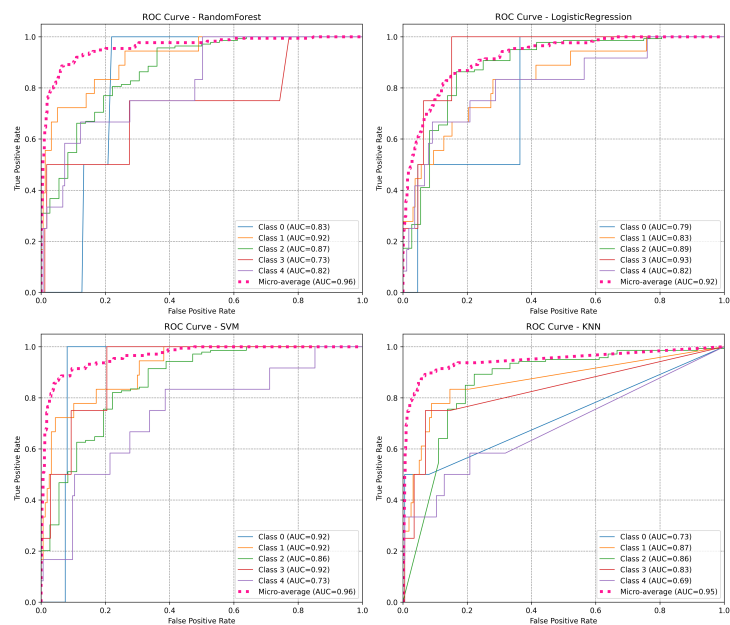


Figure 4.22: ROC Curve for Protein Fold-Based Embeddings

Precision-Recall Curves for Protein Protein Sequence-Based Embeddings

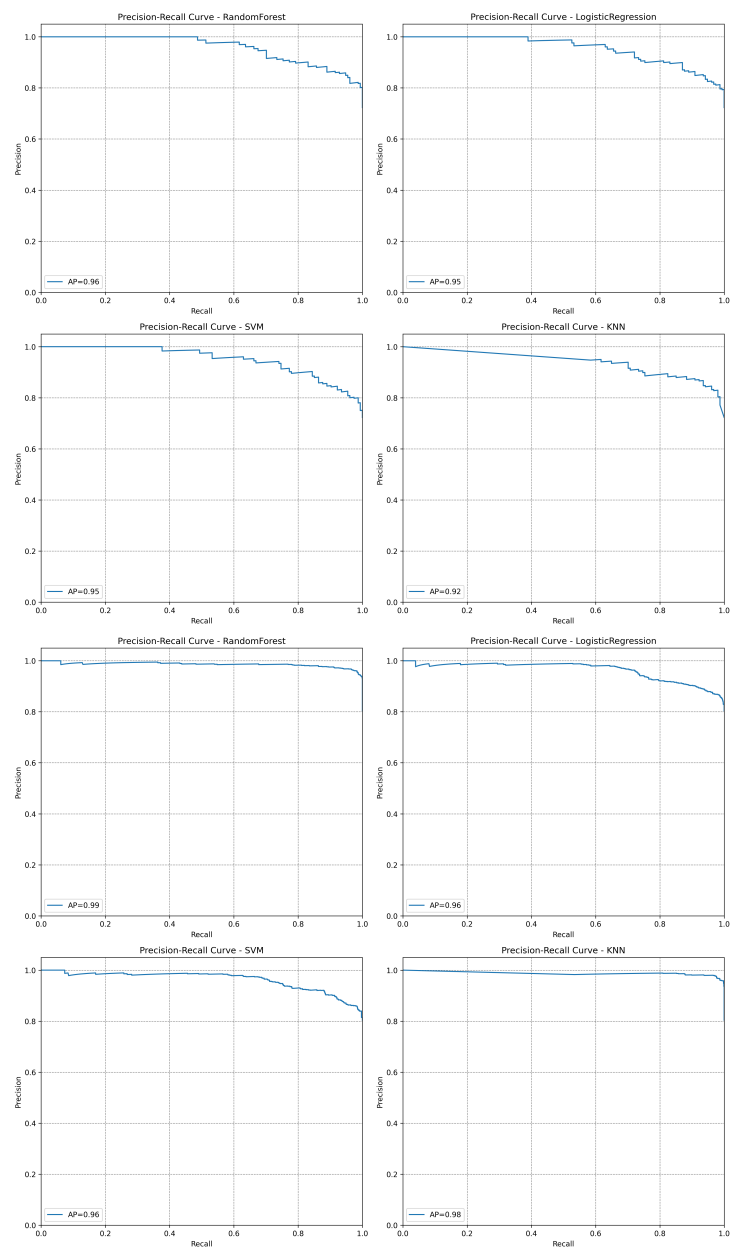


Figure 4.23: Precision-Recall Curves for Protein Sequence-Based Embeddings

Precision-Recall Curves for Protein Protein Fold-Based Embeddings

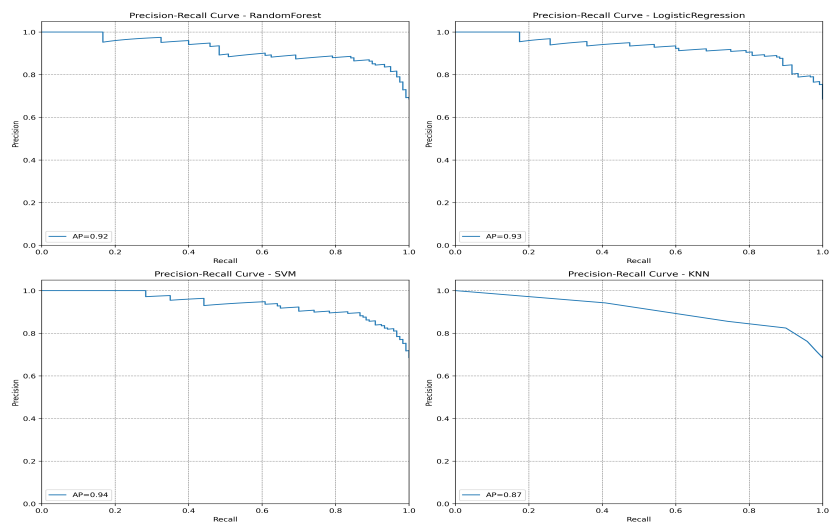


Figure 4.24: Precision-Recall Curve for Protein Fold-Based Embeddings

Chapter 5

Conclusion

Chapter 6

Discussion

Appendix

6.1 Additional Performance Metrics

This section provides additional performance metrics, including Matthews Correlation Coefficient (MCC), Precision, Recall, and F1-score.

Bibliography

- [1] James S. Henkel, Michael R. Baldwin, and Joseph T. Barbieri. Toxins from bacteria. *Medical College of Wisconsin, Department of Microbiology and Molecular Genetics*, 2009. 53151.
- [2] Jeffrey K. Actor. *Basic Bacteriology*. Elsevier, Philadelphia, PA, USA, 2012.
- [3] JR Sheehan, C Sadlier, and B O’Brien. Bacterial endotoxins and exotoxins in intensive care medicine. *BJA Education*, 22(6):224–230, March 12 2022.
- [4] Ciamak Ghazaei. Advances in the study of bacterial toxins, their roles and mechanisms in pathogenesis. *Malaysian Journal of Medical Sciences*, 29(1):4–17, 2022.
- [5] Matthew Silbergleit, Adrian A. Vasquez, Carol J. Miller, Jun Sun, and Ikuko Kato. Oral and intestinal bacterial exotoxins: Potential linked to carcinogenesis. In *Progress in Molecular Biology and Translational Science*, volume 173, pages 129–157. Elsevier, 2020.
- [6] Kaori Ohtani and Tohru Shimizu. Regulation of toxin production in *clostridium perfringens*. *Toxins*, 8(7):207, 2016.
- [7] Nicholas J Loman, Chrystala Constantinidou, Jacqueline ZM Chan, Mikhail Halachev, Martin Sergeant, Charles W Penn, Esther R Robinson, and Mark J Pallen. High-throughput bacterial genome sequencing: an embarrassment of choice, a world of opportunity. *Nature Reviews Microbiology*, 10:599–606, 2012.
- [8] Elizabeth Jane Richardson and Mick Watson. The automatic annotation of bacterial genomes. *Briefings in Bioinformatics*, 14(1):1–12, Jan 2013.

- [9] Mahesh Batta. Machine learning algorithms - a review. *International Journal of Science and Research (IJSR)*, 9(1):167–170, 2020. Research-Gate Impact Factor (2018): 0.28, SJIF (2018): 7.426.
- [10] Authors unknown (replace with the actual author(s)). Machine learning: Algorithms, real-world applications and research directions. *SN Computer Science*, 2(160), March 2021.
- [11] Amer F. A. Alnuaimi and Tasnim H. K. Albaldawi. An overview of machine learning classification techniques. *Open Access Journal Article*, 2021.
- [12] Sudipto Saha and Gajendra P. S. Raghava. Btxpred: Prediction of bacterial toxins. *in Silico Biology*, 7:405–412, January 2007.
- [13] Jorge F. Beltrán, Lisandra Herrera-Belén, Fernanda Parraguez-Contreras, Jorge G. Farías, Jorge Machuca-Sepúlveda, and Stefania Short. Multitoxpred 1.0: A novel comprehensive tool for predicting 27 classes of protein toxins using an ensemble machine learning approach. *Journal Name (If Available)*, 2021.
- [14] Shallu Kotwal, Priya Rani, Tasleem Arif, Jatinder Manhas, and Sparsh Sharma. Automated bacterial classifications using machine learning based computational techniques: Architectures, challenges and open research issues. *Archives of Computational Methods in Engineering*, 29(4):1–22, October 2021.
- [15] Tanja Krüger, Ivan Koludarov, Maria Littmann, Burkhard Rost, and Luisa F. Jiménez-Soto. Toxin data quality: A critical examination of bacterial exotoxins and animal toxins. *Journal Name (If available)*, May 2024.
- [16] Martin Steinegger and Johannes Söding. Mmseqs2 enables sensitive protein sequence searching for the analysis of massive data sets. *Nature Biotechnology*, 35(11):1026–1028, 2017.
- [17] Ahmed Elnaggar, Michael Heinzinger, Christian Dallago, Ghalia Rihawi, Yu Wang, Llion Jones, Tom Gibbs, Tamas Feher, Christoph Angerer, Martin Steinegger, Debsindhu Bhowmik, and Burkhard Rost. Prottrans: Towards cracking the language of life’s code through self-supervised deep learning and high performance computing. *arXiv*, 2021.

- [18] Michael Heinzinger, Konstantin Weissenow, Joaquin Gomez Sanchez, Adrian Henkel, Martin Steinegger, and Burkhard Rost. Prott5: Bilingual language model for protein sequence and structure. *bioRxiv*, 2023.
- [19] Michael Greenacre, Patrick J. F. Groenen, Trevor Hastie, Alfonso Iodice D’Enza, Angelos Markos, and Elena Tuzhilina. Principal component analysis. *Nature Reviews Methods Primers*, 2(100), 2022.